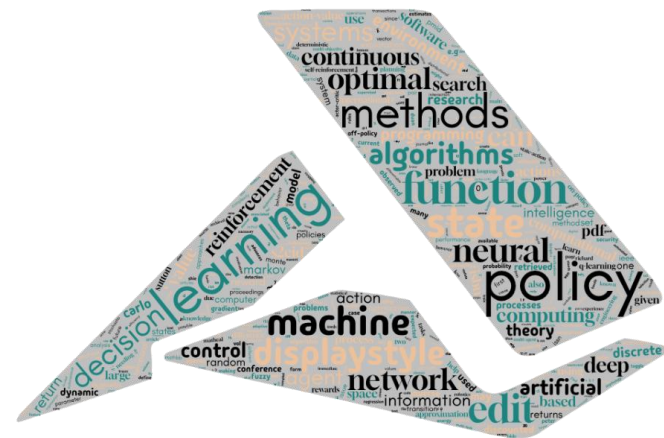
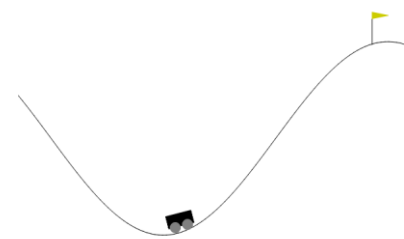




Reinforcement Learning Algorithms and Applications

Chapter 10. DDPG for Mountain Car Continuous

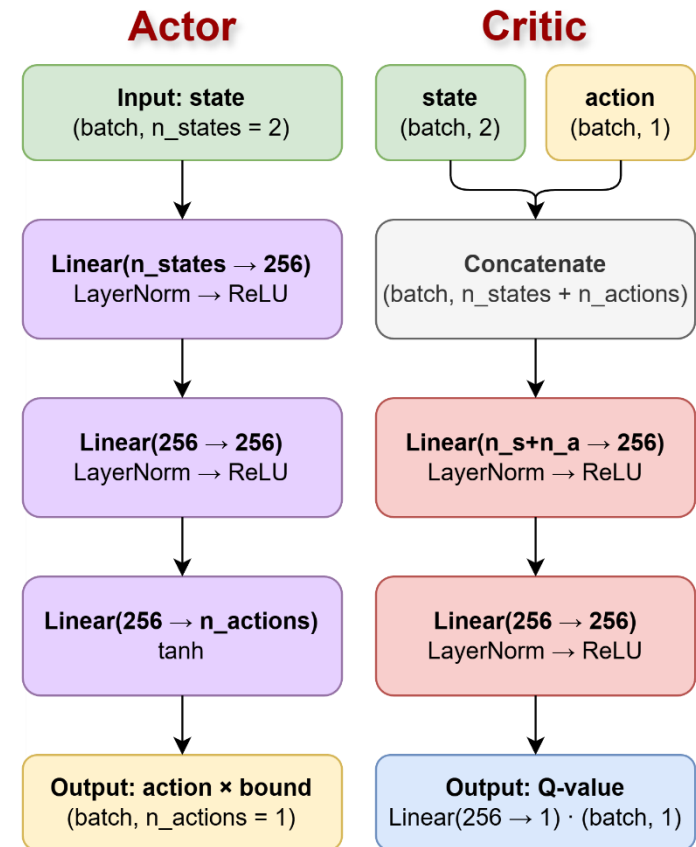
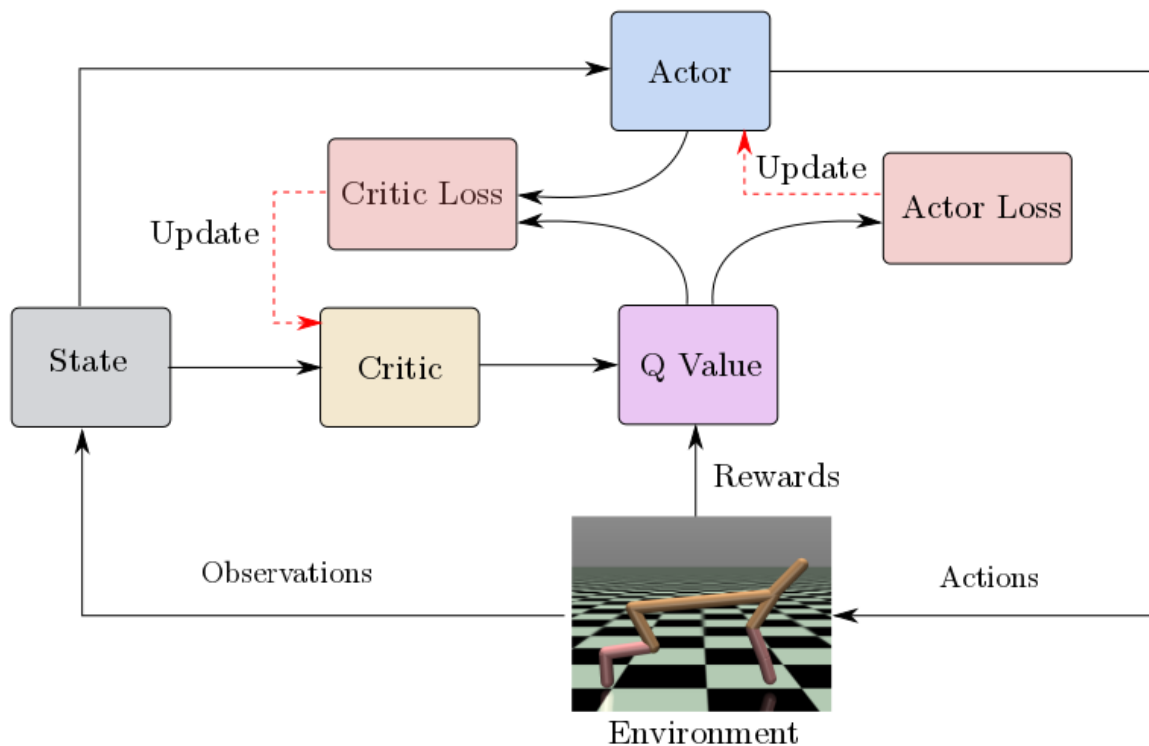
Mountain Car Continuous



- ▶ **Mountain Car Continuous** is a classic **reinforcement learning** environment from the **Gymnasium** library.
- ▶ The agent controls a car stuck in a **valley** between two **hills** and must reach the **flag** at the top of the right hill.
- ▶ The car's **engine** is not strong enough to drive directly up the steep hill, requiring a **momentum-building strategy**.
- ▶ The agent must learn to **oscillate** back and forth to build up enough **kinetic energy** to reach the goal.
- ▶ **State space** consists of two continuous values: **position** (ranging from -1.2 to 0.6) and **velocity** (ranging from -0.07 to 0.07).
- ▶ **Action space** is a single continuous value representing the **force** applied to the car (ranging from -1.0 to 1.0).
- ▶ The goal is to reach a **position** ≥ 0.45 , which represents the **flag location** at the top of the right hill.
- ▶ This environment is particularly challenging because it requires **long-term planning** and **strategic exploration** rather than greedy actions.
- ▶ The **physics simulation** makes this problem ideal for testing **continuous control** algorithms like **DDPG**.
- ▶
$$R = -0.1a^2 + \begin{cases} 100, & x \geq 0.45 \\ 0, & \text{otherwise} \end{cases}$$

Deep Deterministic Policy Gradient (DDPG)

► Actor-Critic Architecture



Ornstein-Uhlenbeck Noise Process

► Motivation

- Gaussian noise causes unstable, jerky actions
- OU noise provides **smooth and realistic exploration**

► OU Noise Model

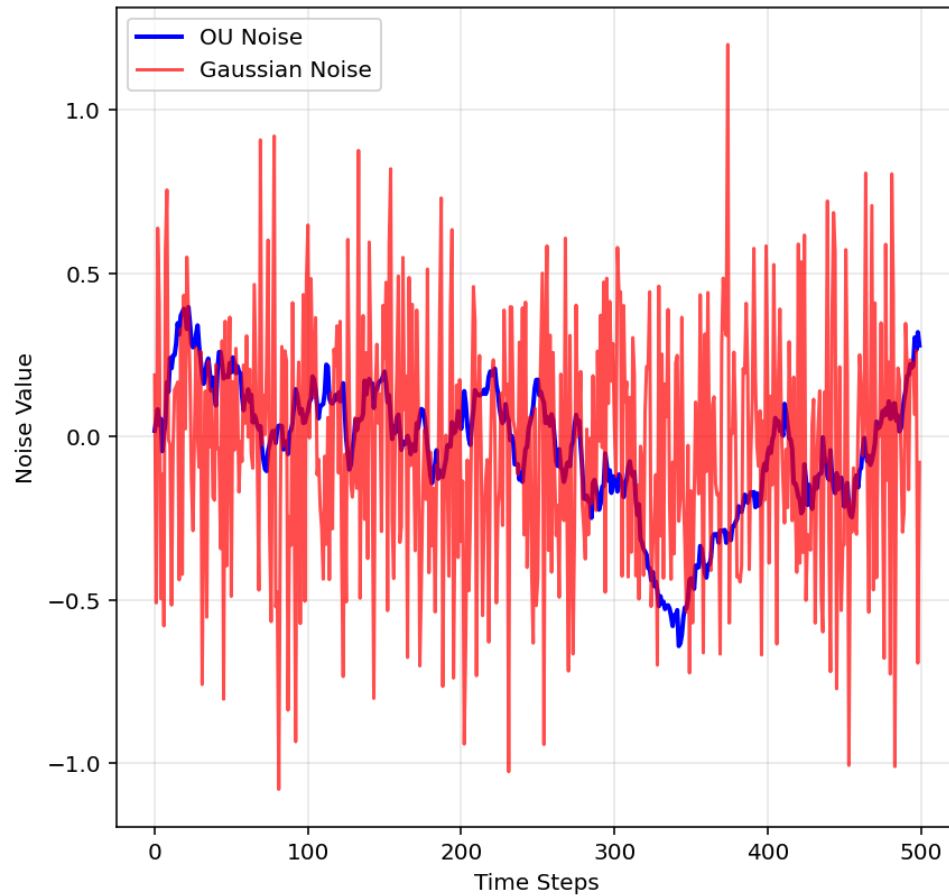
- $$x_{t+1} = x_t + \theta(\mu - x_t)\Delta t + \sigma\sqrt{\Delta t} \mathcal{N}(0,1)$$

► Parameter Meaning

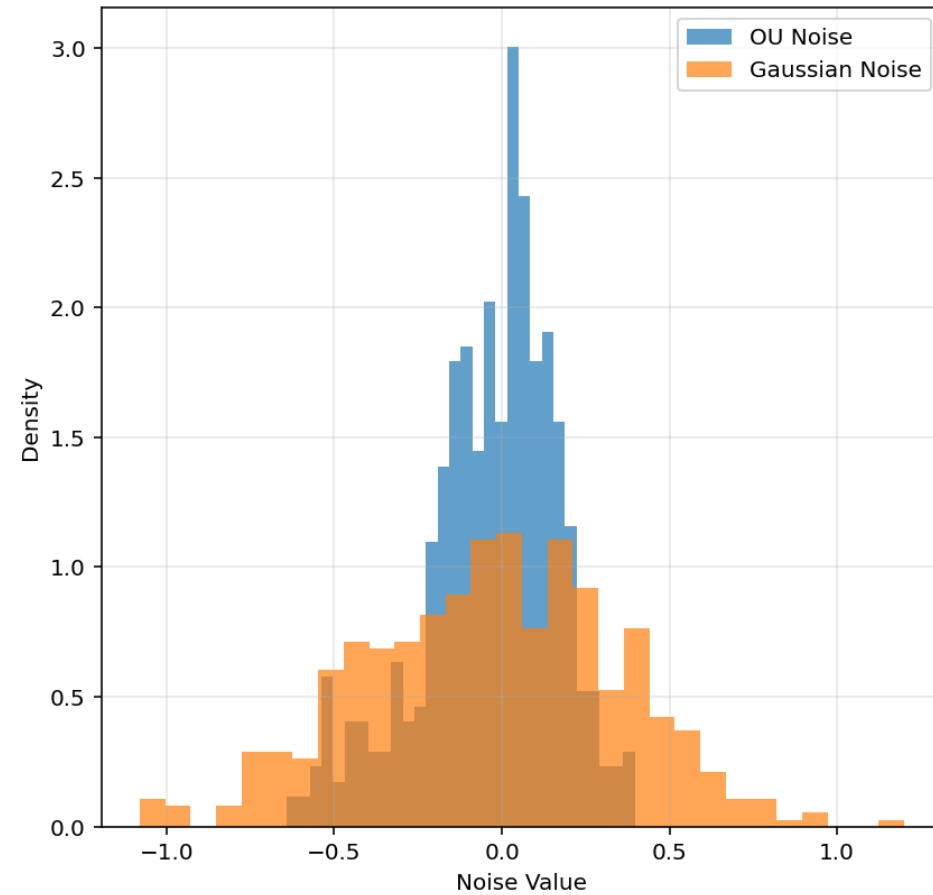
- μ (**mean=0.0**): long-term average value (target level)
- θ (**mean reversion rate=0.15**): how fast noise returns to μ
- σ (**volatility=0.4**): randomness strength (exploration intensity)
- Δt (**time step=0.01**): controls smoothness of change

Ornstein-Uhlenbeck Noise Process

OU Noise vs Gaussian Noise



Distribution Comparison



DDPG Training Algorithm

- ▶ **Actor-Critic** architecture with separate target networks for stability.
- ▶ **Deterministic policy gradient** updates the actor to maximize expected Q-value.
- ▶ **Experience replay** stores transitions $(s, a, r, s', \text{done})$ in replay buffer.
- ▶ **Soft target updates** slowly blend main networks with target networks using $\tau = 0.002$.
- ▶ **Batch size** of 64 with warmup period of 500 steps before learning begins.
- ▶ **Learning rates**: Actor LR = 2×10^{-4} , Critic LR = 1×10^{-3} .
- ▶ **Discount factor** $\gamma = 0.99$ for future reward estimation.
- ▶ **Gradient clipping** prevents exploding gradients during training.

Critic Update Process

- ▶ **Target Q-value** computation using Bellman equation:
- ▶ $y = r + \gamma Q_{target}(s', \mu_{target}(s'))$
- ▶ **Current Q-value** from online critic network: $Q(s, a)$.
- ▶ **Critic loss** using Mean Squared Error:
- ▶ $L_{critic} = \frac{1}{N} \sum_{i=1}^N (y_i - Q(s_i, a_i))^2$
- ▶ **Target networks** provide stable learning targets without moving targets problem.
- ▶ **Done mask** ($\sim \text{done_batch}$) ensures terminal states have zero future value.
- ▶ **Gradient clipping** to 1.0 prevents unstable training.

Actor Update Process

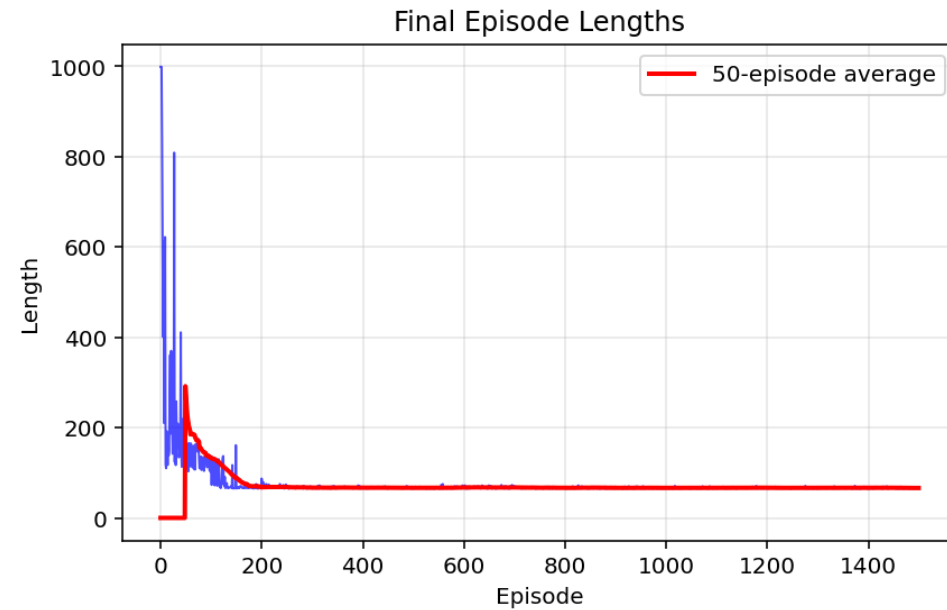
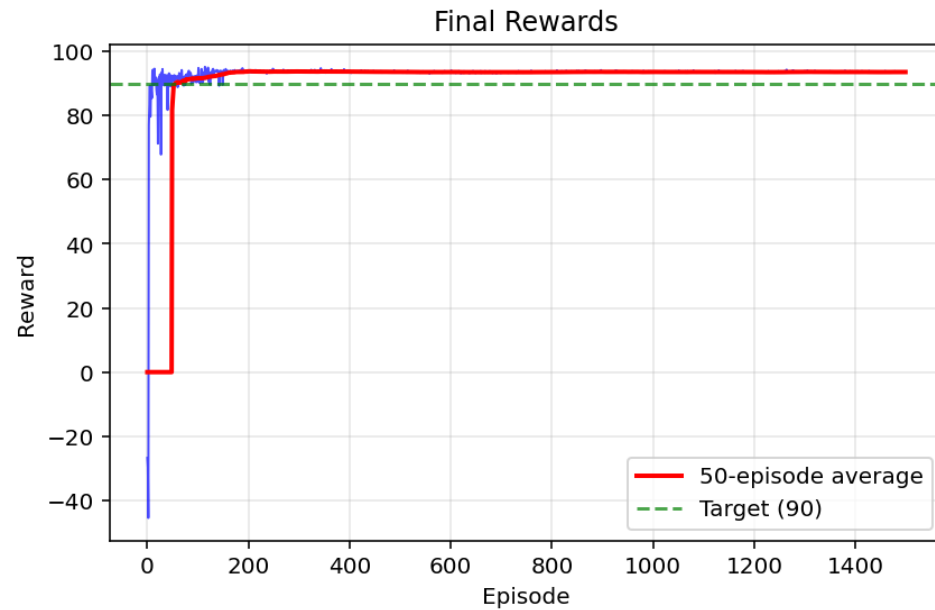
- ▶ **Deterministic Policy Gradient** maximizes expected Q-value:
 - ▶ **Actor loss** (negative because optimizers minimize):
 - ▶ $L_{actor} = -\frac{1}{N} \sum_{i=1}^N Q(s_i, \mu(s_i))$
- ▶ **Chain rule** propagates gradients through critic to actor.
- ▶ Updates actor parameters to produce actions that **maximize Q-values**.
- ▶ **Gradient clipping** maintains training stability.
- ▶ **Target network soft updates** after each training step.

DDPG Training Algorithm

Algorithm 1 Deep Deterministic Policy Gradient

```
1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\theta_{\text{targ}} \leftarrow \theta$ ,  $\phi_{\text{targ}} \leftarrow \phi$  Set target parameters equal to main parameters
3: repeat
4:   Observe state  $s$  and select action  $a = \text{clip}(\mu_{\theta}(s) + \epsilon, a_{\text{Low}}, a_{\text{High}})$ , where  $\epsilon \sim \mathcal{N}$  Select action with noise
5:   Execute  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal Obtain next state, reward, and done signal.
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$  Store experiences
8:   If  $s'$  is terminal, reset environment state.
9:   if it's time to update then
10:    for however many updates do
11:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$  Select a batch of samples
12:      Compute targets
13:       $y(r, s', d) = r + \gamma(1 - d)Q_{\phi_{\text{targ}}}(s', \mu_{\theta_{\text{targ}}}(s'))$  Calculate target Q value
14:      Update Q-function by one step of gradient descent using
          
$$\nabla_{\phi} \frac{1}{|B|} \sum_{(s, a, r, s', d) \in B} (Q_{\phi}(s, a) - y(r, s', d))^2$$
 Train Q network
15:      Update policy by one step of gradient ascent using
          
$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} Q_{\phi}(s, \mu_{\theta}(s))$$
 Train policy network
16:      Update target networks with
          
$$\begin{aligned} \phi_{\text{targ}} &\leftarrow \rho \phi_{\text{targ}} + (1 - \rho) \phi \\ \theta_{\text{targ}} &\leftarrow \rho \theta_{\text{targ}} + (1 - \rho) \theta \end{aligned}$$
 Soft update target network
17:    end for
18:  end if
19: until convergence
```

Training Result

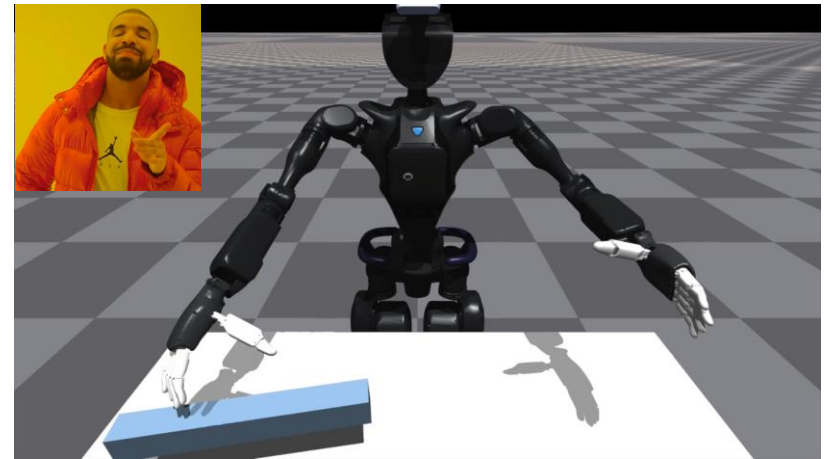
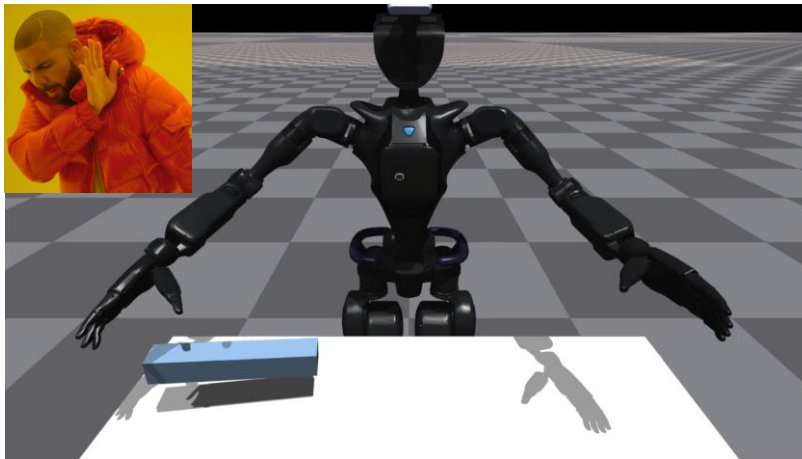


Soft Target Network Updates

- ▶ **Polyak averaging** for stable target network updates:
- ▶ $\theta_{target} \leftarrow \tau\theta + (1 - \tau)\theta_{target}$
- ▶ Where $\tau = 0.002$ is the soft update coefficient.
- ▶ **Main networks** learn quickly while **target networks** change slowly.
- ▶ Prevents **moving target problem** that destabilizes training.
- ▶ Applied to both **actor_target** and **critic_target** networks.
- ▶ **Exponential moving average** ensures smooth convergence.
- ▶ **Small** τ values provide more stable but slower target updates.

Guiding Reinforcement Learning

- ▶ **Reward Hacking**
 - ▶ Exploits flaws in reward design
 - ▶ High reward but wrong behavior



<https://arxiv.org/pdf/2502.20396>

- ▶ **Reward Shaping**
 - ▶ Adjust reward to guide behavior
 - ▶ Faster convergence, less random exploration
- ▶ **Policy Initialization**
 - ▶ Start from a reasonable policy
 - ▶ Improves stability and learning speed

Reference

- ▶ M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- ▶ A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- ▶ S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.